

RESEARCH INTELLIGENCE PLATFORM

AI Research Intelligence & Research Paper Evaluation Platform

PHASE 0 — MASTER PROJECT BLUEPRINT

AI / ML · NLP · RAG · Scholarly Retrieval · Research Intelligence

DOCUMENT STATUS Phase 0 — Blueprint	VERSION v0.1 (Foundational)
DOCUMENT PURPOSE Single source of truth	PROJECT CATEGORY Applied AI / ML System
PRIMARY TRACK AI/ML & NLP Engineering	AUTHOR Project Owner — 3rd-Year CSE

DOCUMENT PURPOSE

This document establishes the foundation of the project. It does not contain implementation code and does not begin Phase 1. Every later phase is expected to remain consistent with this blueprint unless a deliberate, documented architectural change is made.

Table of Contents

(This field updates automatically when the document is opened in Microsoft Word. If it appears empty, right-click it and choose "Update Field.")

Table of Contents	2
1. Executive Project Overview	3
2. Real-World Problem	3
3. Proposed Solution	4
4. What Exactly Does the ML/AI System Do?	5
5. Research Quality Evaluation Framework.....	6
6. System Components.....	7
7. Language and Technology Stack.....	8
8. Technology Significance	10
9. Dataset and Data Strategy.....	10
10. Complete Data Flow	11
11. Complete User Flow	12
12. System Architecture	12
13. Project Repository Structure	13
14. Project Phases and Milestones.....	14
15. Deliverable Checklist	15
16. Skills I Will Learn.....	16
17. Placement Value.....	16
18. Reliability and Evaluation	17
19. Security.....	18
20. Responsible AI	18
21. Scalability Path	19
22. Future Scope.....	20
23. Research Paper Potential	20
24. GitHub Presentation Strategy.....	21
25. Project Success Criteria	21
26. Risks and Mitigation	22
27. Final Project Definition	22

1. Executive Project Overview

Field	Detail
Project Name	Research Intelligence Platform
Category	Applied AI/ML System
Type	Full-stack ML + NLP + IR system
Target Users	Researchers, students, reviewers
Core Objective	Evidence-grounded paper evaluation
Status	Phase 0 — Blueprint

One-Paragraph Description

The Research Intelligence Platform is an AI-assisted system that reads an uploaded research paper, extracts its structure and claims, retrieves related literature from real scholarly sources, and evaluates the paper across multiple measurable dimensions — problem formulation, novelty, methodology, experimental rigor, citation integrity, and reproducibility. Every score the system produces is expected to be traceable to retrieved evidence and stated reasoning, rather than being an unexplained number from a language model.

Value Proposition

IN ONE SENTENCE

Turn an uploaded paper into a structured, evidence-backed research quality assessment, grounded in retrieved literature rather than a model's unverified opinion.

Key Differentiator

Most “chat with your PDF” tools stop at summarization and question answering over a single document. This platform is explicitly designed around retrieval-grounded evaluation: it does not answer questions about the paper in isolation, it compares the paper against external literature, tries to verify whether claims are supported, and produces a structured, multi-dimensional assessment with visible evidence and confidence.

How This Differs From Adjacent Tools

Category	What it typically does	Why this project is different
Chat with PDF	Q&A and summarization over one uploaded file	Adds literature retrieval, claim verification, and structured multi-dimension scoring
Generic LLM summarizer	Produces a free-text summary of the document	Extracts structured research fields (claims, methods, datasets) and grounds them in evidence
Generic plagiarism checker	String/text-similarity matching against a corpus	Focuses on conceptual novelty and claim support, not text-matching originality detection
Literature search engine	Returns a ranked list of related papers	Uses retrieval as one stage feeding claim verification and comparative analysis

Positioning Statement

RESEARCH DECISION-SUPPORT, NOT A SCIENTIFIC AUTHORITY

The platform is positioned as an assistant that helps a researcher self-review and understand how a paper relates to existing literature. It is not positioned as a system that can certify scientific validity, novelty, publishability, or the absence of plagiarism.

2. Real-World Problem

Researchers — particularly students and early-career authors — routinely face a set of recurring, well-understood difficulties when producing and reviewing research papers. This section states those problems plainly, without exaggeration, and separates what is genuinely hard today from what AI assistance can realistically help with.

Problem	Existing problem	Why existing approaches are insufficient	Opportunity for AI assistance
Literature volume	Reviewing sufficient prior work manually is slow, and it is easy to miss relevant papers.	Keyword search and manual reading do not scale with publication volume.	Semantic retrieval across large scholarly indexes surfaces conceptually related work, not just keyword matches.
Differentiation clarity	Authors often struggle to state precisely how their contribution differs from prior work.	Self-assessment of novelty is subjective without a comparison baseline.	Embedding-based comparison against retrieved papers gives a concrete, inspectable basis for the claim.
Unsupported claims	Some claims in a draft are not clearly backed by the paper's own evidence or citations.	Manual claim-by-claim auditing is tedious and easy to skip under deadline pressure.	Automated claim → evidence matching can flag claims worth a second look.
Citation relevance	Citations are sometimes only loosely related to the point being made.	There is no scalable way to check citation relevance by hand across a long reference list.	Semantic similarity between citation context and cited-paper abstract can flag weak matches.
Methodological gaps	Missing baselines or weak experimental design are sometimes only caught late.	Self-review is limited by the author's own blind spots.	A structured methodology checklist compared against retrieved similar papers can prompt reflection.
Reproducibility signals	Reviewers and readers often cannot tell whether a paper is reproducible from the text alone.	There is no lightweight, automatic reproducibility signal today.	Extracting stated code/data/hyperparameter availability gives a partial, explainable signal.
Pre-submission self-review time	A thorough self-review pass takes significant time researchers often do not have.	Manual review does not scale to multiple drafts or tight deadlines.	A first structured pass can triage where a human should focus limited review time.

SCOPE HONESTY

These are real, common friction points — not existential crises in research. The platform is framed as reducing friction and surfacing evidence a researcher can evaluate, not as solving peer review or replacing human judgment.

3. Proposed Solution

The platform implements the following transformation for every uploaded paper:

Input → Processing → Intelligence → Evaluation → Recommendation → User Action

End-to-End Example

A researcher uploads a machine learning paper as a PDF. The platform then performs the following sequence:

- Parses the PDF and identifies its structure (title, abstract, sections, references).
- Extracts claims, methods, datasets, experiments, and the reference list.
- Retrieves related literature from scholarly sources using semantic search.
- Compares the paper's stated contribution against the retrieved literature.
- Attempts to verify citation and claim support against available evidence.
- Evaluates methodology and experimental rigor against the retrieved similar work.

- Estimates novelty and research-gap strength relative to what was retrieved.
- Produces multi-dimensional scores, each with evidence and reasoning attached.
- Identifies risks such as weak baselines or thin citation support.
- Generates evidence-backed, actionable recommendations for the researcher.

DESIGN INTENT

Every step above is meant to be individually inspectable: the researcher can see what was retrieved, what was matched, and why a given score was produced — instead of receiving an opaque single number.

4. What Exactly Does the ML/AI System Do?

This section defines the concrete technical responsibilities of the system, broken down by task type. Each capability is explicitly labeled by the technique category that primarily drives it.

Capability	Primary technique	What it actually does
Document intelligence	ML-based	Document classification, section identification, and layout-aware information extraction over the parsed PDF, so the system knows which text is the abstract, methods, results, etc.
NLP extraction	ML-based	Named entity extraction, research-claim extraction, method extraction, dataset extraction, and result extraction using transformer-based sequence labeling and structured generation.
Embeddings	ML-based	Dense vector representations of text (claims, abstracts, chunks) that place semantically similar content near each other in vector space, enabling similarity search rather than keyword matching.
Semantic similarity	ML-based	Cosine similarity (or similar metrics) between embeddings is used for paper-to-paper comparison and for matching a claim against candidate supporting evidence.
Information retrieval	Retrieval-based	Scholarly search over external sources (e.g. OpenAlex, Semantic Scholar, Crossref, arXiv) to find papers related to the uploaded paper's topic and claims.
Reranking	ML-based	Initial retrieval (e.g. via embeddings or BM25) is often noisy; a cross-encoder or LLM-assisted reranker reorders the top candidates for relevance before they are used downstream.
RAG (retrieval-augmented generation)	Retrieval + LLM reasoning	Retrieved passages are supplied to the LLM as grounding context so its explanations reference actual retrieved evidence instead of relying purely on parametric knowledge.
Claim verification	Retrieval + LLM reasoning	A claim is matched against retrieved evidence passages; the LLM assesses whether the evidence supports, contradicts, or is insufficient for the claim, with a stated confidence.
Citation analysis	Rule-based + ML-based	Reference parsing and resolution are largely rule-based; assessing whether a citation's context is topically relevant to the cited work uses semantic similarity.
Novelty analysis	Retrieval + ML-based (estimation)	Novelty is estimated by how semantically close the paper's contribution is to the nearest retrieved prior work — explicitly a relative estimate, not an absolute judgment.
Research-gap analysis	Retrieval + LLM reasoning	Clusters of retrieved literature are examined for under-covered sub-topics near the paper's area, surfaced as candidate gaps for the researcher to evaluate.
Quality scoring	Rule-based aggregation over ML/retrieval signals	Multiple measurable signals (extraction completeness, evidence support rate, similarity spread, etc.) are combined into the dimension scores defined in Section 5.

Capability	Primary technique	What it actually does
Recommendation generation	LLM reasoning, grounded in prior stages	Identified weaknesses (unsupported claims, missing baselines, thin related-work coverage) are converted into specific, actionable suggestions rather than generic advice.
Confidence estimation	Rule-based + ML-based	Each important assessment carries a confidence signal derived from evidence quantity/quality and retrieval agreement, so the user can see where the system is less certain.

ON NOVELTY AND CITATION VERIFICATION

“Novelty” and “claim verification” are estimation tasks bounded by what the retrieval layer can find. The system reports novelty as a relative, evidence-based estimate against retrieved literature — never as an absolute scientific judgment — and reports claim verification as supported / contradicted / insufficient-evidence, never as a definitive truth value.

5. Research Quality Evaluation Framework

INITIAL PROPOSED EVALUATION FRAMEWORK

The dimensions and any illustrative weights below are a starting design, not a scientifically validated instrument. They are expected to be refined once compared against expert human evaluations in later phases.

Dimension	What it measures
Problem formulation	Is the research problem clearly stated and well-motivated?
Novelty	How does the contribution compare to the nearest retrieved prior work?
Literature quality	Is the related-work coverage reasonably comprehensive and current?
Methodology	Is the method clearly described and appropriate for the stated problem?
Experimental rigor	Are baselines, ablations, and comparisons present and reasonable?
Results quality	Are results clearly reported with appropriate context (not cherry-picked)?
Citation integrity	Are citations topically relevant to the statements they support?
Reproducibility	Are code, data, and hyperparameters described or made available?
Writing clarity	Is the paper's structure and language clear and well-organized?
Research impact (estimated)	How significant might the contribution be, based on retrieved context?

Every Score Must Carry

Evidence · Reasoning · Sources · Confidence · Limitations

Why Arbitrary LLM Scores Are Unacceptable

A single number produced directly by prompting an LLM (“Rate this paper's novelty from 1–10”) is not reproducible, not explainable, and not comparable across papers. Instead, each dimension score is intended to be derived from measurable underlying signals (retrieval results, extraction completeness, claim-verification outcomes), with the LLM used to reason over and explain those signals — not to invent them.

Path to Validation

The framework is expected to be validated by comparing system-generated dimension scores against scores or feedback given by human reviewers (e.g. faculty, senior students, or domain experts) on a small benchmark set of papers, and adjusting the framework based on observed disagreement.

6. System Components

Every major system component is defined below using a consistent Purpose → Input → Processing → Output structure.

Component	Purpose	Input	Processing	Output
Frontend	Give the researcher a usable interface	User interactions, uploaded files	Renders forms, calls the API	UI screens, uploaded paper, rendered report
Authentication	Identify and protect user accounts	Credentials / OAuth tokens	Verifies identity, issues session tokens	Authenticated session
Paper upload service	Accept and validate uploaded PDFs	PDF file	File-type and size validation, virus/malware scan, storage handoff	Stored file reference, upload record
Document processing pipeline	Convert raw PDF into structured text	Raw PDF bytes	Text/layout extraction, OCR fallback if scanned	Structured page/section text
OCR layer	Handle scanned or image-only PDFs	Page images	Optical character recognition	Extracted text with lower confidence flag
Structure extraction	Identify paper sections	Structured page text	Section-boundary detection (title, abstract, methods, etc.)	Section-labeled document
NLP information extraction	Pull out research fields	Section-labeled document	Claim / method / dataset / result extraction	Structured research-info record
Embedding service	Represent text as vectors	Text chunks, claims, abstracts	Transformer embedding model inference	Dense vector representations
Vector retrieval	Find semantically similar content	Query embedding	Approximate nearest-neighbor search	Ranked candidate matches
Scholarly search service	Query external literature sources	Topic / query text	Calls to OpenAlex / Semantic Scholar / Crossref / arXiv APIs	Candidate related papers
Citation analysis engine	Assess reference relevance	Parsed references, citation contexts	Citation-context to cited-abstract similarity	Per-citation relevance signal
Claim verification engine	Check claim support	Claims, retrieved evidence	Claim-evidence matching, LLM-assisted judgment	Supported / contradicted / insufficient label
Similarity engine	Compare papers	Paper embeddings	Pairwise / nearest-neighbor similarity	Similarity scores, nearest papers
Novelty analysis engine	Estimate relative novelty	Similarity results	Distance-to-nearest-prior-work estimation	Novelty estimate with evidence
Research-gap analysis engine	Surface under-covered areas	Retrieved literature cluster	Topic clustering, coverage-gap heuristics + LLM reasoning	Candidate research gaps
Quality scoring engine	Combine signals into dimension scores	All upstream signals	Rule-based aggregation per Section 5 framework	Dimension scores with evidence
Recommendation engine	Turn weaknesses into actions	Scores, flagged issues	LLM-assisted, template-guided generation	Actionable recommendation list
RAG assistant	Answer grounded follow-up questions	User question, retrieved context	Retrieval + LLM generation with citations	Grounded answer with sources

Component	Purpose	Input	Processing	Output
Knowledge graph layer (future)	Model entities and relations	Extracted entities/claims	Graph construction and querying	Queryable research knowledge graph
Database	Persist structured application data	Records from all services	Relational storage, indexing	Queryable structured data
Object / file storage	Store raw and processed files	PDFs, generated reports	Blob storage with access control	Retrievable file objects
Background job processing	Run long tasks asynchronously	Processing jobs/tasks	Queue-based worker execution	Job status, completed results
Monitoring / logging	Observe system health	Application events, metrics	Structured logging, metrics collection	Dashboards, alerts
Evaluation framework	Measure system quality over time	Benchmark inputs, outputs	Automated + human-in-the-loop evaluation	Evaluation reports
API layer	Expose functionality to clients	HTTP requests	Routing, validation, orchestration	JSON responses

7. Language and Technology Stack

For each major technology, the rationale follows a consistent structure: why it is needed, what role it plays, why it fits this project specifically, and what the realistic alternative is.

Technology	Why needed	Role it plays	Why appropriate here	Alternative considered
Python	Primary language for ML, NLP, data pipelines, and backend	Dominant ecosystem for ML/NLP tooling (PyTorch, Hugging Face, spaCy)	Directly needed for every ML component; matches the project's core work	None practical for this project's ML surface area
TypeScript / JavaScript	Frontend application language	Type safety and strong tooling for a React/Next.js interface	Keeps the frontend maintainable as the report UI grows in complexity	Plain JavaScript (weaker safety), or a server-rendered template approach
SQL	Query language for the relational database	Structured queries over structured application data	PostgreSQL is the system of record; SQL is the natural interface	No practical alternative; an ORM abstracts but still emits SQL
FastAPI (Python)	Backend API framework	Async-friendly, typed, fast to develop, natural fit with the Python ML stack	Lets the API layer sit close to the ML/NLP services without a language boundary	Django REST Framework (heavier), Flask (less async-native)
React / Next.js	Frontend framework	Component-based UI, strong ecosystem, good fit for a data-rich report/dashboard UI	Handles the multi-panel evidence/score dashboard described in Section 11	Vue/Svelte (smaller ecosystem for this use case)
PyTorch	Deep learning framework	Underlies most modern NLP/embedding models	Needed to run and, later, fine-tune transformer models	TensorFlow (less common in current NLP tooling)
Hugging Face Transformers	Pretrained model access and inference	Standard library for loading and running transformer models	Fastest path to working extraction/embedding models without training from scratch	Direct model-provider SDKs only (less flexible)

Technology	Why needed	Role it plays	Why appropriate here	Alternative considered
sentence-transformers	Sentence/document embedding models	Purpose-built for producing semantically meaningful embeddings	Directly needed for similarity search and claim-evidence matching	Raw transformer pooling (more manual, less optimized)
spaCy	Lightweight NLP utilities	Fast, well-tested for tokenization/NER pipelines	Useful for lower-level text processing that does not need a full LLM	NLTK (older API, less production-oriented)
PyMuPDF	PDF text/layout extraction	Fast, reliable PDF parsing with layout information	First-line extraction for the majority of digitally-native PDFs	pdfplumber (used as a secondary/table-focused tool)
GROBID	Scholarly PDF structure extraction	Purpose-built for parsing academic paper structure and references	Directly matches the need to detect sections, references, and metadata in research papers	Custom rule-based section detection (less robust)
OCR (pytesseract + pdf2image)	Handle scanned/image PDFs	Necessary fallback when a PDF has no extractable text layer	Ensures the pipeline degrades gracefully instead of failing on scanned uploads	Cloud OCR APIs (viable later for higher accuracy)
PostgreSQL	Primary relational database	Mature, reliable, and supports the pgvector extension	Lets structured data and vector search live in one system for the MVP	MySQL (weaker vector-search story)
pgvector	Vector similarity search inside PostgreSQL	Adds nearest-neighbor vector search to the existing relational database	Avoids operating a separate vector database before it is justified by scale	Dedicated vector DB (Qdrant/Pinecone/Weaviate) once volume justifies it
LLM API (provider-agnostic)	Reasoning, explanation, and grounded generation	Used for tasks that need language understanding beyond pattern matching	Applied narrowly — on top of retrieved evidence — not as the sole source of truth	Self-hosted open-weight LLM once cost/privacy requirements justify it
OpenAlex / Semantic Scholar / Crossref / arXiv	External scholarly data sources	Provide real literature metadata, abstracts, and citation graphs	Needed for retrieval, comparison, and citation checks to be grounded in real papers	Domain-specific scholarly APIs, added where relevant
Docker	Containerization	Consistent environments across development and deployment	Keeps the multi-service architecture reproducible without heavy orchestration	Bare virtual environments (harder to reproduce)
Git / GitHub	Version control and collaboration	Industry-standard source control	Required for the portfolio/GitHub goal of the project	None practical
CI/CD (GitHub Actions)	Automated testing and build checks	Catches regressions before they reach the main branch	Demonstrates MLOps/software-engineering discipline for placement value	Manual testing only (not scalable, weaker signal to reviewers)
Cloud deployment (single small instance to start)	Hosting for a running demo	Makes the project usable and demoable, not just runnable locally	A modest single-instance deployment is sufficient for an MVP	Multi-region/distributed deployment, deferred to Section 21

DELIBERATELY EXCLUDED FROM THE MVP

Kubernetes, a dedicated vector database, microservice-per-feature decomposition, custom foundation-model training, and multi-database architectures are not included in the MVP. They are revisited only if a specific, demonstrated need arises (see Section 21).

8. Technology Significance

Technology	Role	Why Used	ML/Engineering Skill Demonstrated
FastAPI	API layer	Async Python API that connects UI, ML services, and database	Backend API design, async programming
React / Next.js	Frontend	Renders upload flow, dashboards, evidence views	Frontend engineering, state management
PyTorch + Transformers	ML/NLP core	Runs extraction and embedding models	Applied deep learning, model inference
sentence-transformers	Embeddings	Produces vectors for similarity and retrieval	Representation learning, semantic search
GROBID	Document structuring	Extracts sections, references, metadata from PDFs	Applied document AI, tool integration
PostgreSQL + pgvector	Data + vector store	Stores structured data and enables similarity search	Database design, vector search fundamentals
Scholarly APIs	Retrieval	Supplies real related-paper candidates	Information retrieval, API integration
LLM (via API)	Reasoning layer	Explains evidence, drafts recommendations	Prompt engineering, RAG system design
Docker	Environment consistency	Reproducible local/deployed environments	DevOps fundamentals, containerization
GitHub Actions	CI/CD	Automated linting/testing on each change	MLOps discipline, automated testing

9. Dataset and Data Strategy

NO UNVERIFIED DATA CLAIMS

No dataset below is claimed to be “definitely used” until its access terms and format have been verified during Phase 1–2. This section describes realistic candidates and how they would be used, not confirmed commitments.

Category	Description
Development data	A small, manually collected set of sample papers (e.g. authors' own drafts, publicly available papers) used to build and debug the pipeline end-to-end.
Training data	Only needed if/when a component is fine-tuned (e.g. a claim-extraction model); candidate sources include public scholarly-NLP datasets, used only if their license permits it.
Evaluation data	A curated benchmark set of papers with known characteristics (e.g. papers with publicly discussed strengths/weaknesses) plus, where possible, expert annotations for comparison.
External scholarly data	Metadata, abstracts, and citation graphs pulled live from OpenAlex, Semantic Scholar, Crossref, and arXiv at query time, rather than a static bulk download initially.

Category	Description
User-uploaded data	The papers researchers upload for analysis; treated as private user data with access control, not used to train shared models without explicit consent.

Data Handling Pipeline

Collect → License check → Clean → Dedupe → Normalize → Chunk → Embed → Version

Benchmark Construction (Later Phase)

An evaluation benchmark could eventually be built by selecting a modest set of papers (e.g. 20–50), having one or more domain-knowledgeable reviewers score them against the Section 5 framework, and comparing those scores against the system's outputs to measure agreement. This is explicitly a later-phase activity (Section 14, Phase 14) and is not assumed to exist yet.

10. Complete Data Flow

PDF → Validate → Parse → Structure → Extract → Embed → Retrieve → Verify → Evaluate → Score → Report

Stage	Input	Processing	Output	Mode	Failure possibilities
File validation	Uploaded file	Type/size checks, malware scan	Accepted or rejected upload	Sync (fast)	Corrupt file, unsupported type, oversized file, malware flag
Parsing	Validated PDF	Text/layout extraction via PyMuPDF; OCR fallback	Raw structured text	Async (queued)	No extractable text layer, malformed PDF
Structure detection	Raw structured text	GROBID-based section/reference detection	Section-labeled document	Async	Non-standard paper format, missing sections
Research information extraction	Section-labeled document	NLP extraction of claims/methods/datasets/results	Structured research record	Async	Extraction model misses or misattributes a field
Embedding generation	Structured record, text chunks	sentence-transformers inference	Vector representations	Async	Very long documents needing careful chunking
Scholarly retrieval	Query embeddings / key terms	Calls to external scholarly APIs + local vector search	Candidate related papers	Async, external I/O	External API downtime or rate limits
Evidence retrieval	Claims, candidate papers	Passage-level retrieval from candidate abstracts/text	Candidate evidence passages	Async	Full text not available for a candidate paper (abstract-only)
Verification	Claims, evidence passages	Claim-evidence matching + LLM-assisted judgment	Supported/contradicted/insufficient labels	Async	Ambiguous or highly technical claims

Stage	Input	Processing	Output	Mode	Failure possibilities
Evaluation & scoring	All upstream signals	Rule-based aggregation per Section 5	Dimension scores with evidence	Async	Sparse evidence lowering confidence
Recommendations	Scores, flagged issues	LLM-assisted, template-guided generation	Actionable recommendation list	Async	Recommendation might be too generic if evidence is thin
Report / dashboard	All outputs	Assembly into a structured report	Rendered report for the user	Sync (read)	N/A

WHY MOST OF THE PIPELINE IS ASYNCHRONOUS

Parsing, extraction, retrieval, and evaluation can each take several seconds to tens of seconds. These stages run as background jobs so the UI stays responsive; the user sees live status updates rather than a blocked request.

11. Complete User Flow

Register → Login → Dashboard → Upload → Processing → Analysis

Quality score · Dimensions · Evidence · Citations · Similar papers · Novelty · Gaps · Risks · Recommendations · AI reviewer · Export

Graceful Handling of Imperfect Conditions

Condition	System behavior
Upload fails	Show a clear, specific error (file type/size/corruption) and allow retry.
PDF is scanned	Automatically fall back to OCR; flag extracted text as lower-confidence.
Structure cannot be detected	Proceed with best-effort section guesses; clearly mark structure as uncertain.
Scholarly retrieval fails	Show cached/partial results if any; state that comparison coverage is limited.
Citations cannot be verified	Mark citation status as “unverifiable” rather than assuming correctness.
Evidence is insufficient	Label the affected claim/dimension as low-confidence rather than guessing.
AI confidence is low	Surface the confidence value directly next to the assessment, not hidden in a tooltip.

NO FALSE CERTAINTY

The interface is designed so that uncertainty is always visible. A low-confidence score is shown as low-confidence, not smoothed over or hidden.

12. System Architecture

The system is organized into the following layers.

Layer	Primary technology	Responsibility
Client layer	React/Next.js web app	User interaction, report visualization

Layer	Primary technology	Responsibility
API layer	FastAPI	Request routing, validation, authentication, orchestration
Application layer	Python services	Business logic connecting pipeline stages
AI/ML layer	Extraction, embedding, LLM-reasoning services	Executes the ML/NLP capabilities in Section 4
Retrieval layer	Vector search + scholarly API clients	Finds related literature and evidence
Data layer	PostgreSQL + pgvector, object storage	Persists structured data, vectors, and files
Infrastructure layer	Docker, background workers, queue	Runs and schedules asynchronous processing
Observability layer	Logging, metrics, evaluation reports	Tracks system health and quality over time

Component Relationships

Frontend ↔ API ↔ Workers ↔ ML services ↔ LLM | Workers ↔ Retrieval ↔ External scholarly APIs | API ↔ PostgreSQL / Vector store / Object storage | All services → Monitoring

Synchronous vs. Asynchronous

Interaction	Mode
Frontend ↔ API (auth, dashboard reads, status polling)	Synchronous
Upload acceptance and validation	Synchronous (fast checks only)
Document processing pipeline (Section 10, stages 2–10)	Asynchronous, queue-based
Report retrieval once complete	Synchronous (read)
Scholarly API calls	Asynchronous, external I/O with timeouts/retries

13. Project Repository Structure

research-intelligence-platform/

Directory	Responsibility
apps/ (web/, api/)	User-facing application code — Next.js frontend and the FastAPI application endpoint (routes, auth, orchestration).
services/	Business-logic services, one directory per pipeline capability: document-processing, retrieval, nlp, evaluation, recommendation.
ml/	Model-centric code: models, training, inference, evaluation, and experiments.
data/ (raw/, processed/, external/)	Data storage conventions, kept out of version control where licensing requires.
pipelines/	Glue code that sequences services into the end-to-end flow in Section 10.
notebooks/	Exploratory analysis.
tests/	Automated tests exercised by CI on every change.
docs/	This blueprint and all subsequent architecture/decision documentation.
scripts/	One-off utility scripts.

Directory	Responsibility
infrastructure/	Docker Compose (and later deployment) definitions.
configs/	Environment and service configuration.
.github/	CI/CD workflows.
README.md / LICENSE	Project overview and licensing.

DESIGN RATIONALE

This structure favors clear service boundaries over premature microservice deployment: services/ and ml/ are organized as separate Python packages that can run inside the same API process for the MVP, and can be split into independently deployed services later without a directory-level rewrite.

14. Project Phases and Milestones

Phase	Name	Objective	Key technologies	Completion criteria
0	Blueprint	Define scope, architecture, and framework	This document	Documented, internally consistent blueprint
1	Research & Problem Validation	Validate the problem statement and prior art	Literature review, competitor review	Documented findings; scope confirmed or adjusted
2	Requirements & Evaluation Framework	Turn the blueprint into concrete requirements	Functional/non-functional requirements, refined Section 5 framework	Signed-off requirements doc
3	System Architecture	Detailed design of APIs, schemas, and service contracts	API contracts, DB schema, sequence diagrams	Reviewed architecture doc
4	Document Processing	Build PDF parsing, OCR fallback, structure detection	PyMuPDF, GROBID, OCR	Reliable text+structure extraction on a sample set
5	Research Information Extraction	Extract claims, methods, datasets, results	Transformers, spaCy	Structured extraction with manual-review accuracy check
6	Scholarly Retrieval	Integrate scholarly APIs and vector search	OpenAlex/Semantic Scholar/Crossref/arXiv, pgvector	Relevant related-paper retrieval on sample queries
7	Citation & Claim Verification	Build claim-evidence matching and citation relevance checks	Embeddings, reranker, LLM reasoning	Verification labels with evidence on sample claims
8	Novelty & Research-Gap Detection	Implement novelty estimation and gap surfacing	Similarity engine, clustering	Novelty/gap outputs reviewed for plausibility
9	Quality Scoring	Implement the Section 5 scoring engine	Rule-based aggregation	Scores produced with evidence for a full sample paper
10	Recommendation Engine	Generate actionable recommendations from flagged issues	LLM-assisted templates	Recommendations reviewed as specific and non-generic
11	RAG Research Assistant	Add grounded Q&A over the paper and retrieved evidence	RAG pipeline	Answers cite retrieved sources

Phase	Name	Objective	Key technologies	Completion criteria
12	AI Peer Review	Simulate structured reviewer perspectives	Multi-perspective LLM prompting over prior stages	Structured review-style output
13	Knowledge Graph	Model entities/relations across processed papers	Graph construction	Queryable graph over a sample corpus
14	Benchmarking & Evaluation	Build and run the evaluation benchmark	Curated papers, human comparison	Documented agreement/disagreement analysis
15	MLOps & Deployment	Containerize, add CI/CD, deploy a demo instance	Docker, GitHub Actions, cloud host	Publicly reachable demo, automated pipeline
16	Research Paper & Portfolio	Document findings; prepare portfolio materials	Writeup, README, demo video	Polished repository and optional research writeup

SCOPE DISCIPLINE

Each phase has a defined completion criterion. A phase is not extended with new capabilities until its own criterion is met; new ideas are logged for a later phase instead of expanding the current one.

15. Deliverable Checklist

Track	Deliverables
Product	- Working upload-to-report flow - Researcher-facing dashboard - Exportable report
ML	- Embedding pipeline - Similarity/novelty estimation - Evaluated extraction quality on a sample set
NLP	- Section detection - Claim/method/dataset/result extraction - Entity extraction
RAG	- Grounded Q&A over retrieved evidence - Source citation in generated answers
Retrieval	- Scholarly API integration - Vector search over embeddings - Reranking of candidates
Database	- PostgreSQL schema - pgvector similarity queries - Migration scripts
Backend	- FastAPI service - Authentication - Background job processing
Frontend	- Upload flow - Processing status view - Evidence and score dashboard
Testing	- Unit tests for core services - Integration test for the end-to-end pipeline

Track	Deliverables
Evaluation	• Section 5 framework implementation • Benchmark comparison report
Security	• File validation and scanning • Authentication/authorization • Secrets management
MLOps	• Dockerized services • CI/CD pipeline • Basic monitoring/logging
Documentation	• This blueprint • Architecture Decision Records • README and setup guide
Deployment	• Demo-accessible deployed instance
Research	• Documented research questions • Benchmark methodology writeup

16. Skills I Will Learn

Area	Specific skills	Priority
Programming	Python, TypeScript, SQL	Must Learn
ML	Model inference, embeddings, similarity, ranking	Must Learn
ML (advanced)	Fine-tuning, model training	Learn Later
NLP	Transformers, information extraction, semantic search	Must Learn
GenAI	LLM prompting, RAG, grounding, structured generation	Must Learn
Data	Data pipelines, preprocessing	Must Learn
Data (advanced)	Dedicated vector database operations	Learn Later
Backend	FastAPI, REST APIs, authentication	Must Learn
Frontend	React / Next.js	Must Learn
MLOps	Docker, CI/CD, monitoring	Must Learn
MLOps (advanced)	Model/version management at scale, distributed serving	Learn Later
Software engineering	Architecture, testing, Git, modularity	Must Learn
Research	Experimental design, benchmarking, literature analysis	Must Learn
Research (advanced)	Formal research-paper writing for publication	Learn Later

SEQUENCING NOTE

The “Must Learn” column reflects what is required to build the MVP (through roughly Phase 10). “Learn Later” items are deferred until the MVP is functioning, to avoid stalling progress on skills that are not yet load-bearing.

17. Placement Value

Role	What part of the project demonstrates it
ML Engineer	Embedding pipeline, similarity/novelty estimation, model inference services
AI/ML Engineer	End-to-end system combining ML components with a production-style API
NLP Engineer	Section detection, information extraction, entity/claim extraction
GenAI Engineer	RAG assistant, prompt design, grounded generation, structured LLM outputs
Applied Scientist	Evaluation framework design, benchmark methodology, error analysis
Software Engineer (AI specialization)	API design, database schema, async job processing, testing
MLOps Engineer	Dockerization, CI/CD pipeline, monitoring, evaluation-before-deployment discipline

Strongest Components for Interviews

- The retrieval-grounded evaluation architecture, since it goes beyond a basic LLM wrapper.
- The evidence → reasoning → confidence scoring design, a concrete answer to “how do you prevent hallucination?”
- The evaluation/benchmarking methodology, since it shows scientific rigor beyond just shipping code.

Resume Bullet Themes (Illustrative, Not Metrics)

- Designed and built a retrieval-grounded research-paper evaluation pipeline combining NLP extraction, embedding-based retrieval, and LLM-assisted claim verification.
- Implemented a multi-dimensional evaluation framework with evidence and confidence attached to every generated score.
- Built an asynchronous processing pipeline with background workers, structured logging, and CI/CD.

NO FABRICATED METRICS

Resume bullets should not claim specific accuracy numbers, user counts, or performance figures until they are actually measured. Replace bracketed placeholders with real numbers only once they exist.

18. Reliability and Evaluation

CORE PRINCIPLE

The system should be evaluated against evidence and human judgments, rather than trusting model outputs as ground truth.

Practice	What it means here
Ground-truth datasets	A small curated benchmark of papers with known characteristics for repeatable evaluation.
Human expert evaluation	Comparing system outputs against scores/feedback from knowledgeable reviewers.
Retrieval evaluation	Checking whether retrieved papers are actually topically relevant (precision/recall on sample queries).
Information extraction evaluation	Manually checking a sample of extracted claims/methods/datasets for correctness.
Citation verification evaluation	Spot-checking flagged citation-relevance issues against manual review.
Scoring calibration	Checking whether dimension scores move sensibly when known-strong vs. known-weak papers are compared.
Hallucination evaluation	Checking whether generated explanations reference only retrieved evidence, not invented facts.

Practice	What it means here
RAG evaluation	Checking whether RAG answers are actually supported by the passages they cite.
Confidence estimation	Checking whether low-confidence flags correlate with cases that are, in fact, harder or sparser in evidence.
Error analysis	Reviewing failure cases by pipeline stage to find where errors concentrate.
Regression testing	Automated tests that catch pipeline output changes on a fixed sample set.
Model evaluation before deployment	Running the evaluation suite before any model/prompt change ships.

19. Security

Risk area	Concern	Recommended mitigation
Malicious PDF uploads	PDFs can embed scripts or exploit parser vulnerabilities.	Sandboxed parsing, up-to-date parsing libraries, malware scanning on upload.
File validation	Unvalidated files can crash or exploit the processing pipeline.	Strict type/size checks and content sniffing before processing.
Malware risks	An uploaded file could carry malware payloads.	Antivirus/malware scanning integrated into the upload service.
Prompt injection inside documents	Text in a paper could attempt to manipulate LLM instructions.	Treat extracted document text as untrusted data, not instructions; use structured prompting and input isolation.
Data leakage	Uploaded papers may be confidential or unpublished.	Per-user access control; no cross-user data sharing without explicit consent.
Unauthorized access	Accounts or data could be accessed without permission.	Authenticated sessions, least-privilege access to storage and database.
API abuse	Automated abuse of endpoints (e.g. scraping, flooding).	Rate limiting and request quotas per user/IP.
Authentication	Weak auth can be bypassed or brute-forced.	Standard hashed-credential or OAuth-based authentication with session expiry.
Authorization	Users could access data or actions beyond their role.	Explicit per-resource authorization checks in the API layer.
Secrets management	API keys/credentials could leak into code or logs.	Environment-based secrets, excluded from version control, rotated periodically.
Rate limiting	Uncontrolled request volume can degrade service or spike cost.	Per-endpoint and per-user rate limits, especially on LLM/scholarly-API calls.
Storage security	Stored files/data could be exposed if misconfigured.	Private-by-default storage buckets, signed URLs for access.
External API security	Calls to scholarly/LLM APIs could leak sensitive data or fail insecurely.	Minimize data sent externally; use HTTPS; handle failures without leaking internals.
Database security	The database is a high-value target for data exposure.	Network isolation, least-privilege database users, parameterized queries.

20. Responsible AI

POSITIONING STATEMENT

The system is research decision-support, not an autonomous scientific authority. It never presents an AI-generated score as objective truth.

Concern	How it is addressed
AI hallucination	Mitigated by grounding explanations in retrieved evidence and flagging when evidence is insufficient rather than filling gaps with unsupported text.
False confidence	Every important assessment carries an explicit confidence signal; the UI must not present low-confidence results as certain.
Bias	Retrieval and scoring can inherit biases present in scholarly indexes (e.g. field, language, publisher coverage); this limitation is disclosed to the user.
Citation errors	Automated citation-relevance checks can be wrong; they are framed as flags for human review, not final verdicts.
False novelty claims	Novelty is reported as a relative estimate against retrieved literature, explicitly not a claim about global scientific novelty.
False plagiarism accusations	The system does not perform plagiarism detection and must not imply that low novelty or high similarity equals plagiarism.
Intellectual property	Uploaded papers remain the researcher's property; the system does not redistribute or expose uploaded content to other users.
Copyright	Retrieved literature is used for comparison metadata/snippets, not full-text redistribution, respecting source licensing.
Research confidentiality	Unpublished or sensitive drafts are handled with the same access controls as any private user data; no default sharing with third parties.
Human oversight	The system is designed to inform a human reviewer's judgment, not to replace the human decision.
Explainability	Every score is expected to trace to evidence, reasoning, and sources, per the Section 5 framework.
Evidence grounding	LLM-generated text is expected to reference retrieved evidence rather than unstructured model knowledge wherever a factual claim about the paper or literature is made.
Model limitations	The system's outputs are bounded by retrieval coverage and extraction accuracy; these limitations are surfaced to the user, not hidden.

21. Scalability Path

Stage	Focus	Description
Stage 1 — Student MVP	Single-machine / small deployment	One deployed instance, PostgreSQL + pgvector, synchronous where simple, a modest background-worker setup.
Stage 2 — Production prototype	Dockerized services + background workers	Containerized services, a real job queue, structured monitoring, still a small number of instances.
Stage 3 — Scalable platform	Distributed processing + caching + scalable retrieval	Horizontally scaled workers, caching of frequent retrieval results, possible migration to a dedicated vector database.

Stage	Focus	Description
Stage 4 — Research intelligence platform	Knowledge graph + large scholarly corpus + specialized models	Persistent knowledge graph, larger indexed corpus, possibly fine-tuned domain-specific models.

Concern	Approach
Database scaling	Read replicas and connection pooling before considering sharding.
Vector search scaling	pgvector first; migrate to a dedicated vector database only once query volume/latency justifies it.
Worker scaling	Horizontal scaling of stateless background workers behind the job queue.
Model serving	Batched inference and, later, a dedicated model-serving layer if load requires it.
Caching	Cache frequent scholarly-API and retrieval results to reduce external calls and latency.
Queue systems	Start with a simple managed queue; revisit only if throughput demands more.
Storage	Object storage with lifecycle policies for processed artifacts and reports.
Observability	Centralized structured logging and metrics from Stage 2 onward.

NO PREMATURE DISTRIBUTION

Distributed infrastructure is deliberately not introduced in the MVP. Each stage above is adopted only when a concrete, observed bottleneck justifies it.

22. Future Scope

Near-term

- Figure/table interpretation for extracting results from non-text elements
- Automated literature-review drafts summarizing a topic cluster
- Expanded scholarly-source coverage for niche domains

Medium-term

- Research knowledge graph across processed papers
- Research-trend analysis over retrieved literature clusters
- Multimodal understanding for equations and diagrams

Long-term

- Personalized research assistant tuned to a researcher's history
- Research-agent workflows that chain multi-step literature tasks
- Domain-specific evaluators tuned per research field
- Automated evidence maps linking claims across many papers

This list is intentionally limited to capabilities that extend the existing architecture (document understanding, retrieval, evidence grounding). Ideas outside that trajectory are not included here.

23. Research Paper Potential

The project's evaluation methodology, not just its implementation, has potential as a research contribution in its own right. This section proposes candidate research questions; it does not claim the project is automatically novel research.

Candidate Research Questions

- How effectively can evidence-grounded AI evaluate research-paper quality compared with human reviewers?
- How accurately can retrieval-based systems identify potentially unsupported scientific claims?
- How effectively can semantic literature comparison identify potential research gaps?

Evidence Required to Support Any of These

- A benchmark set of papers with independent human review scores for comparison.
- A defined agreement metric (e.g. correlation between system and human scores) computed and reported.
- Documented error analysis explaining where and why the system disagrees with human reviewers.
- Honest reporting of negative results, not only favorable comparisons.

NO ASSUMED NOVELTY

Building the system does not, by itself, constitute a research contribution. A contribution requires the benchmark, the comparison methodology, and the resulting analysis described above.

24. GitHub Presentation Strategy

This Phase 0 document should be committed as a PDF inside the repository's documentation directory, alongside supporting architecture and research materials, structured as follows:

docs/ → phase-0-blueprint.pdf, architecture/, research/, evaluation/, decisions/

How Future Phases Should Reference This Blueprint

Each subsequent phase document should open with a short note confirming it remains consistent with this blueprint, or explicitly documenting any deviation as an Architecture Decision Record (ADR) with its rationale.

Recommended Ongoing Documentation

- Architecture Decision Records (ADRs) for any deliberate deviation from this blueprint.
- Experiment logs for model/prompt/pipeline changes and their observed effect.
- Model cards describing each model in use, its purpose, and its known limitations.
- Dataset documentation covering source, license, and preprocessing for any dataset used.
- Evaluation reports from each benchmarking pass.
- A changelog tracking notable changes per version.
- A living roadmap reflecting the current state of Section 14's phase plan.

25. Project Success Criteria

NO INVENTED NUMERIC TARGETS

Where a numeric target cannot yet be justified, this section states how it should be established later (e.g. from Phase 14 benchmarking) rather than inventing a number now.

Success type	Criteria
MVP success	A researcher can upload a paper and receive a structured report covering the Section 5 dimensions, each with visible evidence, within the pipeline described in Section 10.

Success type	Criteria
Technical success	The pipeline runs end-to-end without manual intervention on a range of typical (digitally-native) research PDFs, with graceful handling of the edge cases in Section 11.
ML success	Extraction, retrieval, and similarity components perform acceptably on a manually reviewed sample; specific accuracy thresholds are to be defined once a benchmark exists (Phase 14).
Research success	A documented benchmark comparison against human review exists, with honestly reported agreement and disagreement, per Section 23.
Production success	A deployed, demoable instance is reachable and stable enough to walk a reviewer or interviewer through a live example.
Placement success	The repository, documentation, and demo together clearly demonstrate the skills mapped in Sections 16 and 17 to a technical reviewer.

26. Risks and Mitigation

Risk	Probability	Impact	Mitigation
Scope creep	High	High	Fixed phase completion criteria (Section 14); new ideas logged, not immediately built.
LLM hallucination	Medium	High	Evidence-grounded prompting, confidence flags, claim-verification stage before scoring.
Poor PDF extraction	Medium	High	GROBID + OCR fallback; flag low-confidence extraction to the user.
Retrieval quality problems	Medium	Medium	Multiple scholarly sources, reranking, and manual spot-checks during Phase 6.
Citation verification difficulty	High	Medium	Framed as a relevance flag, not a definitive verdict; low-confidence labeling.
Lack of expert labels	Medium	Medium	Start with a small benchmark; recruit available peers/mentors for review where possible.
Computational cost	Medium	Medium	Favor smaller open models where feasible; cache and batch external calls.
API dependency	Medium	Medium	Timeouts, retries, and graceful degradation when scholarly/LLM APIs are unavailable.
Data licensing	Low	High	Verify licensing before use (Section 9); avoid bulk redistribution of external content.
Security	Low	High	Mitigations defined in Section 19, implemented from Phase 4 onward.
Overengineering	Medium	Medium	MVP technology constraints stated explicitly in Section 7 and Section 21.
Insufficient evaluation	Medium	High	Evaluation framework and benchmarking are dedicated phases (2, 14), not an afterthought.
Unrealistic novelty detection	Medium	Medium	Novelty explicitly framed as a relative, evidence-based estimate (Section 4, Section 20).

27. Final Project Definition

Question	Answer
What are we building?	An AI-assisted research intelligence platform that analyzes an uploaded research paper against real scholarly literature and produces an evidence-grounded, multi-dimensional quality assessment.
Why are we building it?	To reduce the friction researchers face in literature review, self-assessment, and pre-submission review, using retrieval-grounded AI rather than an opaque single-document chatbot.
Who is it for?	Researchers, graduate and undergraduate students, and reviewers who want a structured, evidence-based first pass before or alongside human review.
What makes it technically different?	Retrieval-grounded, multi-stage evaluation — document understanding, scholarly retrieval, claim verification, and structured scoring — rather than a single LLM prompt over a PDF.
What makes it valuable for researchers?	Visible evidence, reasoning, and confidence behind every assessment, and honest handling of uncertainty instead of false certainty.
What makes it valuable for my placement portfolio?	It demonstrates ML, NLP, IR/RAG, backend, database, and MLOps skills together in one coherent, explainable system, described in Sections 16 and 17.
What will the first version actually contain?	The pipeline through Phase 10 (Section 14): document processing, extraction, retrieval, claim verification, quality scoring, and recommendations, with a working frontend and deployed demo.
What will NOT be built initially?	The knowledge graph, AI peer-review simulation, multimodal figure/equation understanding, and any distributed infrastructure beyond the Stage 1/2 scalability path (Section 21).
What can it eventually become?	A broader research intelligence platform with a knowledge graph, trend analysis, and domain-specific evaluators, as outlined in Section 22, and potentially the basis of a research paper as outlined in Section 23.

CLOSING NOTE

This blueprint is the single source of truth for the project. Future phases should remain consistent with it unless a deliberate, documented architectural change is made.